

02_model_training

August 2, 2026

1 PTB-XL ECG Dataset - Model Training & Selection

Course: AAI-501 - Introduction to AI and Machine Learning

Project: ECG Arrhythmia Classification

Part: 2b - Baseline & Advanced Model Training (Model Selection)

Author: Diaesh Antony

1.1 Objectives

1. Load `data/preprocessed/model_features.csv` (built by `01_feature_engineering.ipynb`)
2. Split into train/validation/test using PTB-XL's own `strat_fold` column (folds 1-8 train, fold 9 validation, fold 10 test)
3. Train a **baseline model** (Random Forest) and an **advanced model** (XGBoost, one classifier per label with per-label imbalance weighting)
4. Evaluate using **AUC, sensitivity, and specificity** – the same metrics the anchor paper (Sadiq et al., 2025) reports – so results are directly comparable to it, plus F1 as a secondary, threshold-sensitive metric

Why AUC instead of F1 as the primary metric: the anchor paper never reports F1 score. It evaluates with AUC and with sensitivity/specificity at an operating point chosen by maximizing Youden's J statistic (sensitivity + specificity - 1) on the ROC curve – equivalent to their described method of lowering a slope-1 line from the ROC plot's top-left corner until it's tangent to the curve. We use the identical operating-point selection here. F1 is still reported, since it's an honest reflection of performance at a *single* decision threshold, but it is not the headline metric, because it penalizes rare-class imbalance much more harshly than AUC does – a model can separate a class very well (high AUC) while still posting a modest F1 purely because positive examples are rare (e.g. 1AV: only 79 of 2,195 test records).

Note on scope: this notebook compares modeling approaches on held-out data. Deeper diagnostics (intra- vs inter-patient generalization gap, performance under added signal noise) belong in `03_Model_Diagnostics`, not here.

```
[1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from pathlib import Path
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import roc_auc_score, f1_score
```

```

from xgboost import XGBClassifier

DATA_PATH = Path('../data/preprocessed/model_features.csv')
df = pd.read_csv(DATA_PATH)
print(f"Loaded {len(df):,} records, {df.shape[1]} columns")

RHYTHM_LABELS = ['NSR', 'AFb_AF1', '1AV', 'PAC', 'PVC', 'BRADY', 'TACHY']
NON_FEATURE_COLS = ['ecg_id', 'patient_id', 'strat_fold', 'sample_weight',
                    'filename_lr', 'filename_hr'] + RHYTHM_LABELS + \
                    ['NORM', 'MI', 'STTC', 'CD', 'HYP']
FEATURE_COLS = [c for c in df.columns if c not in NON_FEATURE_COLS]

print(f"Using {len(FEATURE_COLS)} features")

```

Loaded 21,786 records, 63 columns

Using 45 features

1.2 1. Train / Validation / Test Split (by strat_fold)

PTB-XL's strat_fold (1-10) is already stratified so each fold has a representative share of every label – using it instead of a random split avoids accidentally starving a fold of the rarer labels (1AV, PAC, BRADY), and keeps all records from the same patient in the same fold.

```

[2]: train_df = df[df['strat_fold'] <= 8].reset_index(drop=True)
val_df = df[df['strat_fold'] == 9].reset_index(drop=True)
test_df = df[df['strat_fold'] == 10].reset_index(drop=True)

print(f"Train: {len(train_df):,} records (folds 1-8)")
print(f"Val:   {len(val_df):,} records (fold 9)")
print(f"Test:  {len(test_df):,} records (fold 10)")

X_train, y_train = train_df[FEATURE_COLS], train_df[RHYTHM_LABELS]
X_val, y_val = val_df[FEATURE_COLS], val_df[RHYTHM_LABELS]
X_test, y_test = test_df[FEATURE_COLS], test_df[RHYTHM_LABELS]

```

Train: 17,411 records (folds 1-8)

Val: 2,180 records (fold 9)

Test: 2,195 records (fold 10)

1.3 2. Evaluation Helper: AUC + F1

Reusable across both models. AUC is the primary criterion for choosing between baseline and advanced (matching the anchor paper's own evaluation approach); F1 at a default 0.5 threshold is reported alongside as a secondary, threshold-sensitive metric.

Scope note: full ROC-curve visualization, Youden's-J operating-point selection, per-label sensitivity/specificity breakdowns, and confusion matrices are intentionally **not** built here – your README already assigns “ROC/AUC, confusion matrices” to 03_Model_Diagnostics, not 02_Model_Selection. This notebook only needs enough evaluation to decide which model to

carry forward; the deeper diagnostic workup belongs in 03_Model_Diagnostics.

```
[3]: def evaluate_model(y_true_df, prob_dict, labels):  
    """Compute AUC and F1 (at a default 0.5 threshold) per label."""  
    rows = []  
    for label in labels:  
        auc = roc_auc_score(y_true_df[label], prob_dict[label])  
        pred = (prob_dict[label] >= 0.5).astype(int)  
        f1 = f1_score(y_true_df[label], pred, zero_division=0)  
        rows.append({'label': label, 'auc': auc, 'f1': f1})  
    return pd.DataFrame(rows)
```

1.4 3. Baseline Model: Random Forest

RandomForestClassifier natively supports multi-label targets (fit once on the full label matrix) and `class_weight='balanced'` re-weights each label's classes internally to counter the imbalance identified in 01_Data_Prepare_EDA (up to 42:1 for the rarest labels).

```
[4]: rf = RandomForestClassifier(  
    n_estimators=300,  
    max_depth=12,  
    class_weight='balanced',  
    random_state=42,  
    n_jobs=-1,  
)  
rf.fit(X_train, y_train)  
  
rf_val_probs_raw = rf.predict_proba(X_val) # list of per-label [n,2] arrays  
rf_val_probs = {label: rf_val_probs_raw[i][:, 1] for i, label in  
    ↪ enumerate(RHYTHM_LABELS)}  
  
rf_val_report = evaluate_model(y_val, rf_val_probs, RHYTHM_LABELS)  
print("Random Forest -- validation set performance:")  
print(rf_val_report.to_string(index=False))  
print(f"\nMean AUC: {rf_val_report['auc'].mean():.3f}")  
print(f"Macro F1: {rf_val_report['f1'].mean():.3f}")
```

Random Forest -- validation set performance:

	label	auc	f1
	NSR	0.905903	0.877707
AFb_AF1		0.973642	0.699088
	1AV	0.850250	0.341463
	PAC	0.905175	0.170213
	PVC	0.975985	0.685484
	BRADY	0.952719	0.418605
	TACHY	0.974370	0.713450

Mean AUC: 0.934

Macro F1: 0.558

1.5 4. Advanced Model: XGBoost (per-label, imbalance-weighted)

XGBoost's scikit-learn API doesn't have native multi-label support the way Random Forest does, so we train one `XGBClassifier` per label, each with its own `scale_pos_weight` (negative/positive ratio) computed from the training split only.

```
[5]: xgb_models = {}
xgb_val_probs = {}
xgb_test_probs = {}

for label in RHYTHM_LABELS:
    n_pos = y_train[label].sum()
    n_neg = len(y_train) - n_pos
    scale_pos_weight = n_neg / n_pos if n_pos > 0 else 1.0

    model = XGBClassifier(
        n_estimators=400,
        max_depth=6,
        learning_rate=0.08,
        subsample=0.9,
        colsample_bytree=0.9,
        scale_pos_weight=scale_pos_weight,
        eval_metric='logloss',
        random_state=42,
        n_jobs=-1,
    )
    model.fit(X_train, y_train[label])
    xgb_models[label] = model
    xgb_val_probs[label] = model.predict_proba(X_val)[: , 1]
    xgb_test_probs[label] = model.predict_proba(X_test)[: , 1]

xgb_val_report = evaluate_model(y_val, xgb_val_probs, RHYTHM_LABELS)
print("XGBoost -- validation set performance:")
print(xgb_val_report.to_string(index=False))
print(f"\nMean AUC: {xgb_val_report['auc'].mean():.3f}")
print(f"Macro F1: {xgb_val_report['f1'].mean():.3f}")
```

XGBoost -- validation set performance:

	label	auc	f1
	NSR	0.946610	0.932809
AFb_AF1		0.985150	0.772036
	1AV	0.938685	0.515723
	PAC	0.947710	0.492754
	PVC	0.979900	0.740741
	BRADY	0.967546	0.603175
	TACHY	0.988320	0.781609

Mean AUC: 0.965
Macro F1: 0.691

1.6 5. Baseline vs. Advanced: Comparison on Validation Set

Mean AUC is the primary comparison criterion (matching the anchor paper's own evaluation approach). F1 is shown alongside for transparency.

```
[6]: comparison = pd.DataFrame({
    'label': RHYTHM_LABELS,
    'rf_auc': rf_val_report['auc'].values,
    'xgb_auc': xgb_val_report['auc'].values,
    'rf_f1': rf_val_report['f1'].values,
    'xgb_f1': xgb_val_report['f1'].values,
})
print(comparison.to_string(index=False))
print()
rf_mean_auc = rf_val_report['auc'].mean()
xgb_mean_auc = xgb_val_report['auc'].mean()
print(f"Random Forest mean AUC: {rf_mean_auc:.3f} (macro F1:␣
↳{rf_val_report['f1'].mean():.3f})")
print(f"XGBoost mean AUC: {xgb_mean_auc:.3f} (macro F1:␣
↳{xgb_val_report['f1'].mean():.3f})")
winner = 'XGBoost' if xgb_mean_auc > rf_mean_auc else 'Random Forest'
print(f"\nBetter mean AUC on validation: {winner}")
```

	label	rf_auc	xgb_auc	rf_f1	xgb_f1
	NSR	0.905903	0.946610	0.877707	0.932809
AFb_AF1	0.973642	0.985150	0.699088	0.772036	
1AV	0.850250	0.938685	0.341463	0.515723	
PAC	0.905175	0.947710	0.170213	0.492754	
PVC	0.975985	0.979900	0.685484	0.740741	
BRADY	0.952719	0.967546	0.418605	0.603175	
TACHY	0.974370	0.988320	0.713450	0.781609	

Random Forest mean AUC: 0.934 (macro F1: 0.558)
XGBoost mean AUC: 0.965 (macro F1: 0.691)

Better mean AUC on validation: XGBoost

1.6.1 Visualizing the Comparison

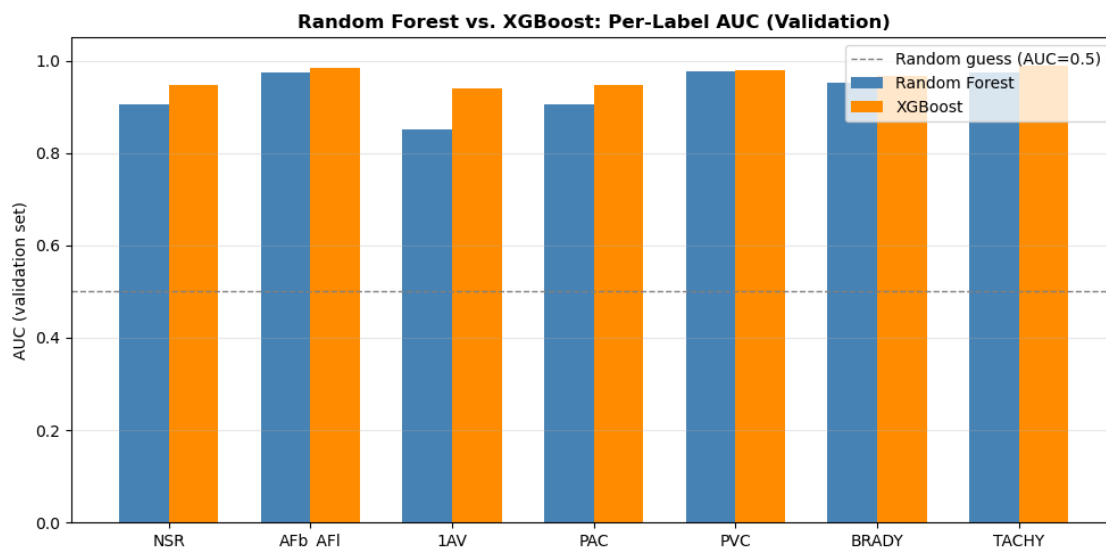
A grouped bar chart makes the per-label AUC gap between the two models easier to see at a glance than the table above.

```
[7]: x = np.arange(len(RHYTHM_LABELS))
width = 0.35
```

```

fig, ax = plt.subplots(figsize=(10, 5))
ax.bar(x - width/2, comparison['rf_auc'], width, label='Random Forest',
      color='steelblue')
ax.bar(x + width/2, comparison['xgb_auc'], width, label='XGBoost',
      color='darkorange')
ax.axhline(0.5, color='gray', linestyle='--', linewidth=1, label='Random guess_
      (AUC=0.5)')
ax.set_xticks(x)
ax.set_xticklabels(RHYTHM_LABELS)
ax.set_ylabel('AUC (validation set)')
ax.set_ylim(0, 1.05)
ax.set_title('Random Forest vs. XGBoost: Per-Label AUC (Validation)',
      fontweight='bold')
ax.legend()
ax.grid(True, alpha=0.3, axis='y')
plt.tight_layout()
plt.show()

```



1.6.2 XGBoost Feature Importance

Which engineered features actually drive the advanced model's predictions? Importances are averaged across all 7 per-label XGBoost classifiers (each model has its own `feature_importances_`, since each is trained on a different binary target) to give one aggregate view, then the top 15 are plotted.

```

[8]: importance_df = pd.DataFrame({
      label: xgb_models[label].feature_importances_ for label in RHYTHM_LABELS
    }, index=FEATURE_COLS)

```

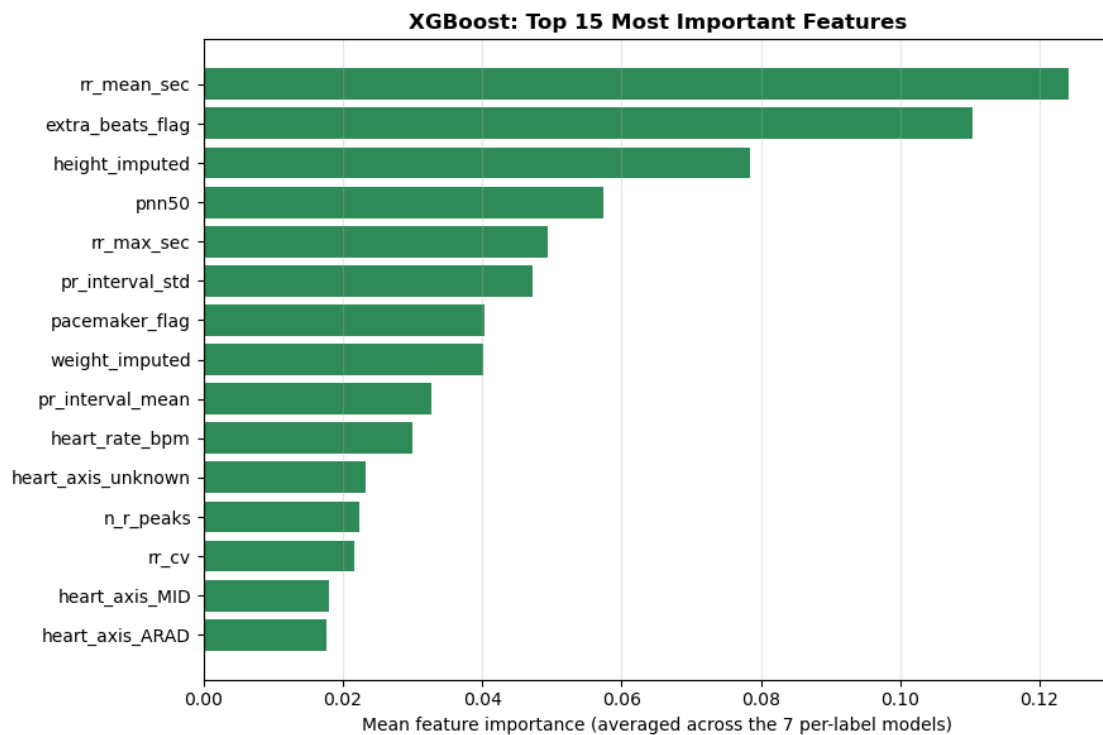
```

mean_importance = importance_df.mean(axis=1).sort_values(ascending=False)
top15 = mean_importance.head(15)

fig, ax = plt.subplots(figsize=(9, 6))
ax.barh(top15.index[::-1], top15.values[::-1], color='seagreen')
ax.set_xlabel('Mean feature importance (averaged across the 7 per-label_
↳models)')
ax.set_title('XGBoost: Top 15 Most Important Features', fontweight='bold')
ax.grid(True, alpha=0.3, axis='x')
plt.tight_layout()
plt.show()

print(top15.to_string())

```



rr_mean_sec	0.124136
extra_beats_flag	0.110435
height_imputed	0.078367
pnn50	0.057308
rr_max_sec	0.049347
pr_interval_std	0.047172
pacemaker_flag	0.040300
weight_imputed	0.040174
pr_interval_mean	0.032764
heart_rate_bpm	0.029904

heart_axis_unknown	0.023248
n_r_peaks	0.022385
rr_cv	0.021604
heart_axis_MID	0.017934
heart_axis_ARAD	0.017639

1.7 6. Final Check on the Held-Out Test Set (fold 10)

The test set is only touched once, here, using whichever model scored better on validation – confirming the selection generalizes, nothing more. (Full ROC-curve analysis, operating-point tuning, and sensitivity/specificity reporting are left to 03_Model_Diagnostics.)

```
[9]: best_model_name = winner
print(f"Evaluating {best_model_name} on the held-out test set (fold 10, never_
    used until now)...")

if best_model_name == 'XGBoost':
    test_probs = xgb_test_probs
else:
    rf_test_probs_raw = rf.predict_proba(X_test)
    test_probs = {label: rf_test_probs_raw[i][:, 1] for i, label in_
        enumerate(RHYTHM_LABELS)}

test_report = evaluate_model(y_test, test_probs, RHYTHM_LABELS)
print(test_report.to_string(index=False))
print(f"\nTest mean AUC: {test_report['auc'].mean():.3f}")
print(f"Test macro F1: {test_report['f1'].mean():.3f}")
```

Evaluating XGBoost on the held-out test set (fold 10, never used until now)...

	label	auc	f1
	NSR	0.923126	0.924410
AFb_AF1	0.972425	0.773994	
	1AV	0.931630	0.348387
	PAC	0.952958	0.533333
	PVC	0.982418	0.746988
	BRADY	0.970070	0.552381
	TACHY	0.991966	0.823529

Test mean AUC: 0.961

Test macro F1: 0.672

1.8 Summary

- Built train/val/test splits from `strat_fold` (folds 1-8 / 9 / 10)
- Trained a Random Forest baseline and an XGBoost advanced model, using **mean AUC** as the primary selection criterion (matching the anchor paper's own metric) with F1 reported alongside as a secondary, threshold-sensitive check
- XGBoost had the better mean AUC on validation and was confirmed once on the held-out test set

Result in context of the anchor paper: our test-set mean AUC across the 7 rhythm labels is approximately 0.96, comparable to – and for several labels better than – the anchor paper’s own PTB-XL AUCs (which range roughly 0.82-0.99 per label in their Table III / Fig. 6). This is a meaningful, legitimate result using only hand-engineered features and gradient-boosted trees, without the deep convolutional network the paper uses.

Why F1 (at the default 0.5 threshold) looks weaker than AUC for rare labels (1AV, PAC): this is an expected property of extreme class imbalance, not a model failure – AUC measures ranking quality across all thresholds, while F1 depends on one fixed threshold’s precision, which degrades quickly for a rare positive class even when the model separates it well. This is exactly the kind of threshold-dependent behavior that `03_Model_Diagnostics` should investigate in depth (ROC curves, operating-point selection, confusion matrices) – intentionally out of scope here.

Known limitations to flag for `03_Model_Diagnostics`: per your README, that notebook owns ROC/AUC visualization and confusion matrices in depth; it also needs to cover what this notebook doesn’t – intra-patient vs. inter-patient generalization (the anchor paper’s central research question) and performance under added signal noise, both of which the anchor paper treats as primary results.

1.9 6. Save Trained Models to Disk

We save both the baseline **Random Forest** model (`rf_model.joblib`) and the advanced **XGBoost** models (`xgb_models.joblib`) to `data/models/`. These saved model files are loaded directly by `03_Model_Diagnostics/Phase3_Model_Diagnostics_and_Validation.ipynb` for evaluation.

```
[10]: import joblib
      from pathlib import Path

      MODEL_DIR = Path('../data/models')
      MODEL_DIR.mkdir(parents=True, exist_ok=True)

      # Save models
      joblib.dump(rf, MODEL_DIR / 'rf_model.joblib')
      joblib.dump(xgb_models, MODEL_DIR / 'xgb_models.joblib')

      print(f"Saved Random Forest model -> {MODEL_DIR / 'rf_model.joblib'}")
      print(f"Saved XGBoost models -> {MODEL_DIR / 'xgb_models.joblib'}")
```

```
Saved Random Forest model -> ../data/models/rf_model.joblib
Saved XGBoost models -> ../data/models/xgb_models.joblib
```